



Санкт-Петербургский государственный университет

Кафедра системного программирования

Распределённая обработка графов: исследование масштабируемости Galois Distributed

Гавриленко Михаил, группа 23.Б15-мм

Санкт-Петербург
2025

Постановка задачи

Цель: исследовать сильное масштабирование BFS, PageRank и SSSP при увеличении числа MPI-процессов в Galois Distributed.

Задачи:

- Сформулировать и проверить гипотезы о масштабируемости
- Провести эксперименты: $P \in \{1, 2, 4, 6, 8\}$ MPI-процессов на 3 структурно разных графах, $T = 2$ OpenMP-потока (фиксировано)

Модель BSP (Bulk-Synchronous Parallel):



Разбиение ОЕС (Outgoing Edge-Cut): вершина v и все её исходящие рёбра $v \rightarrow u_i$ на одном процессе. Если u_i на другом процессе — там создаётся *зеркало* $\tilde{u}_i$.

Коэффициент репликации RF = среднее число копий вершины:

Граф	RF при $P = 2$	RF при $P = 4$	RF при $P = 8$
LiveJournal	1.62	2.47	3.49
RGG (2^{20})	1.000	1.001	1.002
RoadNet-PA	1.007	1.009	1.016

Граф	Вершин	Рёбер	Ср. степень	Диаметр	Тип
soc-LiveJournal1	4.8M	69M	14.2	≈ 5	Социальный
rgg_n_2_20_s0	1.0M	7.3M	7.0	≈ 1200	Геометрический
roadNet-PA	1.1M	3.1M	1.4	≈ 320	Дорожный

- **LiveJournal** — RF растёт с P ; малый диаметр ≈ 5
- **RGG** — $RF \approx 1$ при любом P , но высокий диаметр ≈ 1200
- **RoadNet-PA** — планарный, малая степень, диаметр ≈ 320

Аппаратура:

- Apple M2 Max, 12 ядер, macOS Tahoe
- MPICH 4.3.0
- clang 21.0.0
- $T = 2$ OpenMP-потока на процесс (фиксировано)
- $P \in \{1, 2, 4, 6, 8\}$
- 10 прогонов; run 0 — прогрев, отброшен

Метрики:

- $T(P)$ — время run 1 (мс)
- $T(P)/T(1)$ — нормализованное время
< 1 = быстрее, > 1 = медленнее
- $E(P) = \frac{T(1)}{P \cdot T(P)}$ — эффективность

BFS — алгоритм и ключевой код

Pull-BFS: каждая вершина v проверяет своих соседей; если $\text{dist}(u) + 1 < \text{dist}(v)$ — обновить. Один BSP-раунд = один уровень обхода.

Ключевой оператор (bfs_pull.cpp):

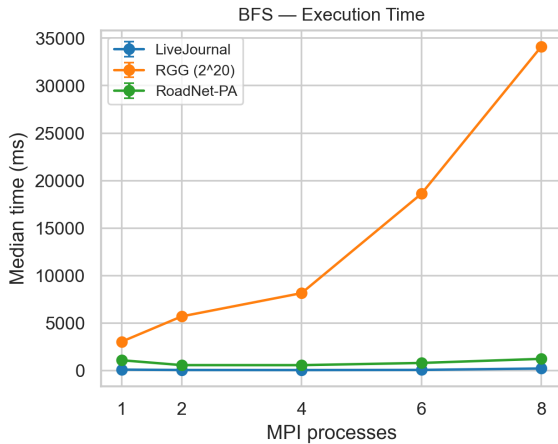
```
void operator()(GNode src) {
    auto& snode = graph->getData(src);
    for (auto jj : graph->edges(src)) {
        GNode dst = graph->getEdgeDst(jj);
        uint32_t new_dist = graph->getData(dst).dist_current + 1;
        if (galois::min(snode.dist_current, new_dist) > new_dist)
            bitset_dist_current.set(src);
    }
}
```

После каждой итерации Gluon рассылает обновлённые значения по зеркалам:

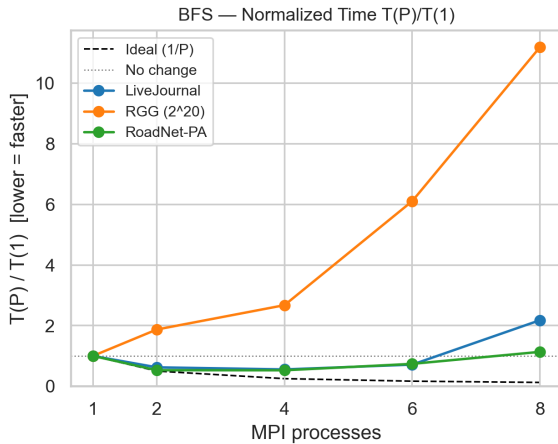
```
syncSubstrate->sync<writeSource, readDestination, Reduce_min_dist_current, Bitset_dist_current>("BFS");
```

- **BFS/LiveJournal:** диаметр ≈ 5 — мало BSP-раундов; основной overhead — RF. Ожидаем небольшое ускорение, затем деградацию.
- **BFS/RoadNet:** ~ 320 BSP-раундов $\times$ стоимость барьера. Ожидаем монотонную деградацию.
- **BFS/RGG:** диаметр ≈ 1200 . Сильный оверхед на коммуникацию из-за кол-ва операций.

BFS — время выполнения ($T = 2$, фиксировано)



BFS — нормализованное время $T(P)/T(1)$



< 1 — ускорение; > 1 — деградация. Пунктир — идеал $1/P$.

BFS — число итераций и время

	$P = 1$	$P = 2$	$P = 4$	$P = 6$	$P = 8$
BFS/LJ итер.	2	2	4	5	5
BFS/LJ $T(P)/T(1)$	1.00	0.77	1.23	2.55	3.89
BFS/RGG итер.	495	929	1226	2377	—
BFS/RGG время (мс)	1557	4202	14836	45067	timeout
BFS/Road итер.	318	328	345	379	447
BFS/Road $T(P)/T(1)$	1.00	0.79	2.78	6.69	11.11

- BFS/LJ при $P=2$: 2 итерации, параллелизм помогает — $T(2)/T(1) = 0.77$
- BFS/RGG: итерации растут в $5\times$ ($P=1 \rightarrow 6$) из-за staleness — суперлинейная деградация

BFS: Push vs Pull на LiveJournal

Push-BFS: активная вершина рассылает обновление соседям.

Pull-BFS: вершина проверяет всех своих соседей сама.

	$P = 1$	$P = 2$	$P = 4$	$P = 6$	$P = 8$
Pull (мс)	53	41	65	135	206
Push (мс)	1	4	24	113	356

- **Push в 50× быстрее при $P=1$:** LJ имеет диаметр ≈ 5 , фронтير мал — Push обрабатывает только активные вершины, Pull всегда сканирует все 69M рёбер
- **При $P=8$ Push медленнее Pull (356 vs 206 мс):** с ОЕС-разбиением исходящие рёбра активных вершин пересекают границы разделов — больше коммуникации

Гипотезы:

- PR/LJ: ускорение при $P=2$ (высокое Work/Comm), деградация при $P \geq 4$ (RF+итерации)
- PR/RGG, RoadNet: $RF \approx 1$, но итерации растут с P

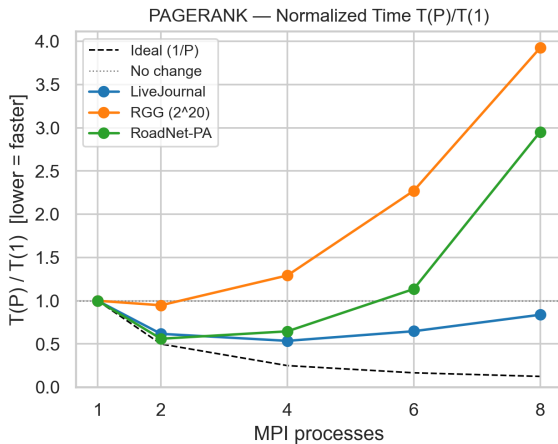
PageRank — число итераций

	$P = 1$	$P = 2$	$P = 4$	$P = 6$	$P = 8$
PR/LJ время (мс)	7326	5760	7710	22772	26121
PR/LJ $T(P)/T(1)$	1.00	0.79	1.05	3.11	3.57
PR/LJ итераций	93	143	160	424	410
PR/RGG $T(P)/T(1)$	1.00	1.35	3.63	11.65	20.55
PR/RGG итераций	51	76	74	189	268
PR/Road $T(P)/T(1)$	1.00	0.95	2.65	8.26	14.60
PR/Road итераций	51	57	64	99	160

PageRank — время выполнения



PageRank — нормализованное время $T(P)/T(1)$



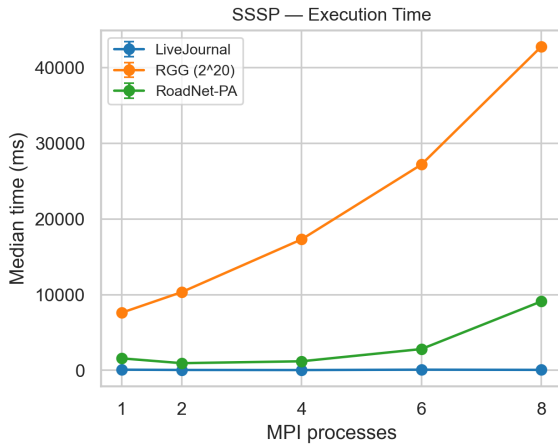
SSSP — результаты

	$P = 1$	$P = 2$	$P = 4$	$P = 6$	$P = 8$
SSSP/LJ $T(P)/T(1)$	1.00	0.79	1.00	1.61	1.85
SSSP/LJ итераций	1	1	1	1	1
SSSP/RGG время (мс)	3868	6618	19084	41781	timeout
SSSP/RGG итераций	878	1000*	1000*	1000*	—
SSSP/Road $T(P)/T(1)$	1.00	0.83	2.57	10.46	28.07
SSSP/Road итераций	375	377	375	374	377

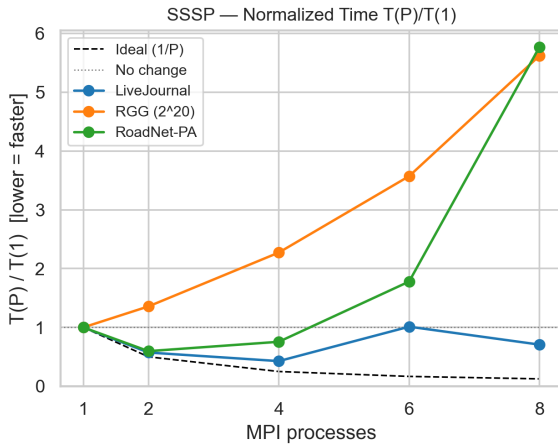
* — достигнут лимит `maxIterations=1000`, алгоритм не сошёлся

- SSSP/LJ: 1 итерация при любом P — power-law граф, короткие пути $\Rightarrow$ сходится за один проход
- SSSP/RGG: не сходится за 1000 раундов при $P \geq 2$ (огромный диаметр)

SSSP — время выполнения



SSSP — нормализованное время $T(P)/T(1)$



Почему число итераций растёт с P ?

Причина — staleness зеркал в BSP-модели:

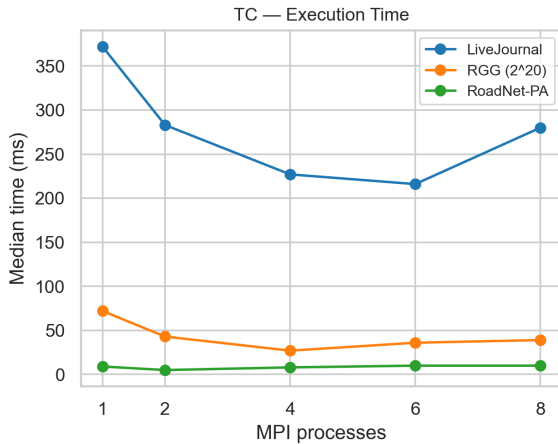
- 1 Мастер v в итерации k читает значение зеркала $\tilde{y}$
- 2 $\tilde{y}$ содержит значение из *предыдущего* раунда — Gluon sync ещё не доставил обновление
- 3 v «не видит» улучшения и пропускает итерацию k
- 4 На итерации $k+1$ зеркало обновлено — v наконец обновляется
- 5 Больше процессов = больше зеркал = больше задержанных обновлений

От чего зависит стоимость синхронизации?

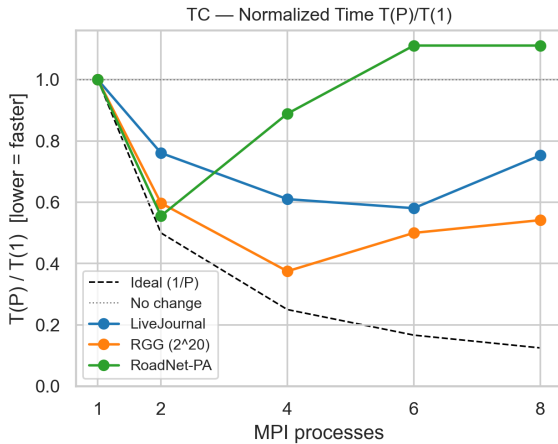
$$\text{Cost(sync)} \propto \underbrace{\text{RF}(P)}_{\text{копий на вершину}} \times \underbrace{|V|}_{\text{вершин}} \times \underbrace{s}_{\text{байт/поле}} \times \underbrace{I(P)}_{\text{итераций}}$$

- **RF(P)**: для LJ $1.0 \rightarrow 1.62 \rightarrow 3.49$ при $P = 1, 2, 8$ — в $3.5\times$ больше данных на итерацию
- **$I(P)$** : PR/LJ $93 \rightarrow 410$ при $P = 1 \rightarrow 8$ — в $4.4\times$ больше раундов sync
- **Суммарно** трафик PR/LJ растёт $\approx 3.5 \times 4.4 \approx 15\times$ от $P=1$ к $P=8$

Triangle Counting



Triangle Counting



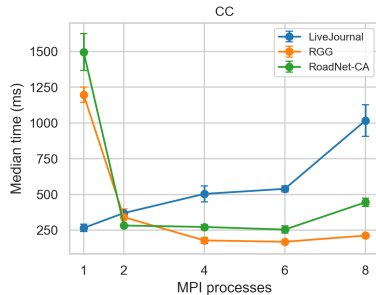
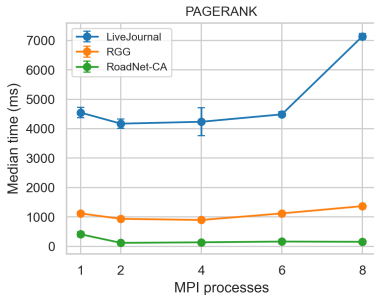
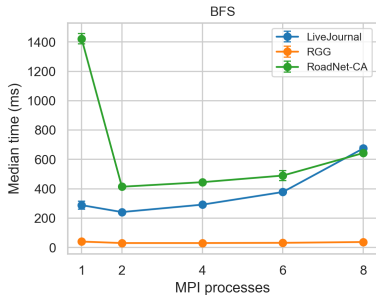
Закключение

Вывод	Подтверждение
BFS и SSSP на LJ ускоряются при $P=2$	$T(2)/T(1) = 0.77$ и 0.79 соответственно
LJ деградирует при $P \geq 4$: RF растёт	RF = 3.49 при $P=8$; $T(8)/T(1) > 3.5$
RGG — вырожденный случай для BFS/SSSP	Итерации BFS: $495 \rightarrow 2377$ ($P=1 \rightarrow 6$); SSSP не сходится при $P \geq 2$
Итерации растут с P (staleness зеркал)	PR/LJ: $93 \rightarrow 410$; BFS/RGG: $495 \rightarrow 2377$
Sync-cost $\propto RF(P) \times I(P)$	LJ: оба множителя растут ($\times 15$ итого)

На реальном кластере (RTT ~ 100 мкс): накладные расходы на барьеры вырастут в 10–100 раз. Эффективны только конфигурации с малым диаметром и $RF \approx 1$.

Справка: время выполнения при $P \times T = \text{const}$

Execution Time vs. Number of MPI Processes



Справка: нормализованное время при $P \times T = \text{const}$

